基于多线程的互联网异构数据采集、数据清洗与挖掘可视化系统

multithread-data-collection-mining-system

作者姓名：_____张馨予_____

时　　间：___2024 年 5 月____

目录

1．引言 ---------- 1
1.1 项目背景 ---------- 1
1.2 系统开发环境 ---------- 1
1.3 定义 ---------- 1
1.4 参考资料 ---------- 1
2．项目概述 ---------- 2
2.1 功能介绍 ---------- 2
2.2 数据来源 ---------- 3
2.3 数据清洗 ---------- 4
3．实现 ---------- 4
3.1 功能模块实现 ---------- 4
3.2 分析结果 ---------- 19
4．项目总结 ---------- 20

1. 引言

1.1 项目背景

随着房地产市场的快速发展，二手房交易已成为房产市场的重要组成部分。深圳作为一线城市，其二手房价格、户型分布、区域差异等数据具有极高的分析价值，可为购房者、房产从业者及市场研究人员提供决策参考。

传统的数据获取方式依赖人工统计，效率低下且易出错。本项目旨在开发一套 “深圳二手房数据爬取与可视化系统”，通过自动化技术爬取安居客平台的深圳二手房数据，实现数据的存储、清洗、统计分析与可视化展示，构建从数据获取到分析应用的全流程解决方案。

本报告详细阐述系统的开发思路、实现过程与分析结果，可供课程实验评估、同类项目开发参考，读者对象为课程教师、相关技术开发者。

1.2 系统开发环境

1.2.1 开发环境配置

操作系统：Windows 10 64 位
开发语言：Python 3.9+
核心库：requests 2.31.0（网络请求）、BeautifulSoup4 4.12.3（HTML 解析）、pandas 2.1.4（数据处理）、matplotlib 3.8.2（可视化）、tkinter（桌面 UI）
开发工具：PyCharm 2023.2.5

1.2.2 系统关联关系

本系统为独立桌面应用，无需依赖其他外部系统，通过网络请求与安居客服务器进行数据交互，本地存储数据文件并完成可视化展示，具有良好的独立性和可移植性。

1.3 定义

本报告涉及的核心术语与缩写说明如下：
1. CSV：即逗号分隔值文件（Comma-Separated Values），是一种轻量的结构化数据存储格式，本项目中用于本地存储爬取的二手房原始数据及清洗后的标准化数据，采用 UTF-8-SIG 编码避免中文乱码问题。
2. HTML：即超文本标记语言（HyperText Markup Language），是网页内容的基础构成语言，本项目通过解析安居客网页的 HTML 代码，提取二手房房源的核心信息（如户型、价格等）。
3. UI：即用户界面（User Interface），本项目中特指基于 tkinter 库开发的桌面可视化操作界面，包含爬取控制按钮、进度条、日志区域、数据可视化展示区等功能模块。
4. 反爬机制：是网站为防止爬虫程序批量获取数据而设置的防护措施，常见形式包括请求头校验、会话验证、访问频率限制等，本项目通过模拟浏览器的 Cookies 和请求头信息，绕

过网站的基础反爬限制。
5. 数据清洗：是对原始爬取数据的优化处理流程，包含去重（删除重复房源记录）、数值提取（从文本中解析有效数值）、缺失值处理（剔除关键信息缺失的无效数据）等操作，最终生成可直接用于分析的高质量数据。

1.4 参考资料

[1] Requests 官方文档. https://requests.readthedocs.io/en/latest/
[2] BeautifulSoup4 官方文档. https://www.crummy.com/software/BeautifulSoup/bs4/doc/
[3] Pandas 官方文档. https://pandas.pydata.org/docs/
[4] Matplotlib 官方文档. https://matplotlib.org/stable/index.html
[5] Python tkinter 编程实战。李刚 著。人民邮电出版社，2022.
[6] 安居客深圳二手房官网. https://shenzhen.anjuke.com/sale/

2. 项目概述

2.1 功能介绍

本系统是一套集“数据爬取、数据存储、数据清洗、数据可视化、交互监控”于一体的桌面应用程序，功能完备、操作便捷，可实现二手房数据从获取到分析展示的全流程自动化处理，核心功能详细说明如下：

数据爬取功能：支持用户自定义设置爬取页码范围（起始页与结束页可灵活输入），通过网络请求自动批量爬取安居客深圳二手房列表页的房源信息。精准提取标题、户型、面积、朝向、楼层、建造年份、小区名称、区域、商圈、地址、标签、总价(万)、单价(元/㎡)等 13 项核心业务字段；采用浏览器请求伪装（Cookies+Headers）和访问频率控制（每页休眠 2 秒），绕过网站基础反爬机制，保证爬取稳定性；同时支持爬取任务的优雅启停，检测到停止信号后，等待当前页面爬取完成再终止，避免数据丢失。

数据存储功能：默认将爬取的原始房源数据以 CSV 格式持久化存储到本地，文件命名为“深圳二手房.csv”；采用 UTF-8-SIG 编码格式，有效解决 Excel 打开 CSV 文件时的中文乱码问题；支持数据追加写入模式，重复启动爬取任务时，不会覆盖已有数据，仅将新爬取的房源信息追加到文件末尾；初始化时自动检测 CSV 文件是否存在，若不存在则自动创建文件并写入预设表头，为后续数据存储提供规范结构。

数据清洗功能：针对原始爬取数据存在的重复记录、格式不统一、缺失值等问题，提供自动化清洗处理。核心流程包括：以“标题+小区名称”为唯一标识去除重复房源；通过正则表达式从文本字段中提取数值（如从“100 ㎡”提取面积数值、从“2000 年建造”提取年份数值），实现格式标准化；将总价、单价等字段转换为数值类型，便于后续统计计算；提取户型中的房间数特征，新增专属字段；删除总价、面积等关键字段缺失的无效数据；最终将清洗后的高质量数据保存为“深圳二手房_cleaned.csv”，为可视化分析提供可靠数据支撑。

数据可视化功能：优先调用清洗后的高质量数据文件，若未找到则自动兼容原始数据文件，提升系统容错性；通过 2×2 布局的 4 类统计图表，直观展示深圳二手房市场的核心数

据特征：一是各区域平均房价横向柱状图，展示前 10 大高价区域的价格分布；二是户型分布柱状图，呈现不同房间数房源的数量占比；三是价格与面积散点图，通过抽样 500 条数据呈现两者的相关性；四是单价分布直方图，清晰展示单价的集中区间；所有图表支持嵌入 tkinter 桌面界面，适配窗口大小缩放，可视化效果直观易懂。

交互与日志功能：基于 tkinter 搭建友好的可视化操作界面，功能分区清晰（控制面板、日志区域、可视化区域）；控制面板支持爬取启停控制、页码参数设置，进度条实时展示爬取进度，状态标签同步更新系统运行状态（就绪/爬取中/已完成）；日志区域以带时间戳的形式，实时记录爬取过程、数据保存条数、清洗结果、错误信息等内容，支持自动滚动到最新日志，便于用户追溯和监控系统运行状态；功能按钮状态联动（爬取中禁用“开始”按钮、启用“停止”按钮），防止误操作，提升使用体验。

2.2 数据来源

本系统的数据来源于安居客深圳二手房交易平台（https://shenzhen.anjuke.com/sale/），该平台是国内知名的房产信息服务平台，提供大量真实、实时的二手房房源数据，具有较高的权威性和实用性。

数据爬取技术实现

请求伪装：针对安居客平台的基础反爬校验机制，系统通过配置完整的 cookies 和 headers 字典实现浏览器请求伪装。其中 cookies 包含用户会话标识（sessid）、设备唯一标识（id58）等关键信息，用于维持与服务器的合法会话状态；headers 包含 User-Agent（声明浏览器类型与版本）、Referer（模拟合法请求来源）、Accept 系列字段（声明支持的响应格式）等，完整还原了普通浏览器的请求特征，成功绕过网站的非登录态反爬校验，避免被判定为恶意爬虫而拒绝服务。

分页爬取：系统深入分析安居客二手房列表页的 URL 构造规则，发现其分页存在明确的格式差异：第 1 页为基础 URL（https://shenzhen.anjuke.com/sale/），无需拼接页码参数；第 2 页及以上的 URL 则遵循 https://shenzhen.anjuke.com/sale/p{page}/的格式，其中{page}为对应页码数值。基于该规则，系统支持用户自定义输入起始页与结束页，自动循环构造对应页码的 URL，实现批量分页爬取，满足用户对不同数据量的获取需求。

HTML 解析：系统采用 BeautifulSoup4 库搭配 html.parser 解析器，实现对网页 HTML 内容的高效解析。首先通过 CSS 选择器 div.property[tongji_tag="fcpc_ersflist_gzcount"]精准定位到单个房源的信息容器节点，批量获取当前页面的所有房源节点；随后针对每个房源节点，通过 select_one 和 select 方法提取对应字段，同时结合特征关键词筛选目标信息——如通过“㎡”识别面积字段、“年建造” 识别建造年份字段、“层”识别楼层字段，以及预设朝向枚举值（南、北、南北等）筛选朝向字段，确保能精准提取到 13 项核心房源信息，同时通过 try-except 异常处理，避免单个房源解析失败导致整页爬取中断。

线程控制：为解决爬取任务阻塞 UI 界面导致的“假死”问题，系统采用多线程编程模式，将爬取任务与 UI 交互任务分离。其中 UI 线程负责响应用户操作（点击按钮、输入页码等）和更新界面状态（进度条、状态标签、日志信息），后台爬取线程（crawl_thread）专门执行数据爬取任务，且设置为守护线程，随主程序退出而自动终止。同时通过 stop_event 线程事件对象实现爬取任务的优雅停止：用户点击“停止”按钮时，仅发送停止信号，后台线程在遍历下一页前检测该信号，若已触发则等待当前页面爬取完成后正常退出，避免数据写入中断或文件损坏，保证数据完整性。

反爬优化：为降低被网站封禁 IP 的风险，系统加入了访问频率控制机制。在每完成一页房源的爬取、保存操作后，通过 time.sleep(2)让程序休眠 2 秒，主动降低请求频率，模拟

普通用户的浏览节奏，避免短时间内大量请求触发网站的高频访问预警机制，进一步提升了爬取过程的稳定性和可持续性。

2.3 数据清洗

原始爬取数据存在格式不统一、重复记录、缺失值等问题，需通过以下步骤进行清洗：

去重处理：以“标题+小区名称”为联合唯一标识，删除重复房源记录。

数值提取：从文本字段中提取数值（如从“100 ㎡”提取 100.0、从“2000 年建造”提取 2000），生成标准化数值列（如面积_数值、建造年份_数值）。

类型转换：将总价、单价等字段转换为数值类型，便于统计计算。

特征提取：从户型字段中提取房间数（如从“3 室 2 厅”提取 3），新增“房间数”字段。

缺失值处理：删除总价、面积等关键字段缺失的记录，保证数据有效性。

结果保存：将清洗后的数据保存为独立 CSV 文件，供可视化分析使用。

3. 实现

3.1 功能模块实现

```
import requests
from bs4 import BeautifulSoup
import csv
import os
import time
import re
import threading
import tkinter as tk
from tkinter import ttk, messagebox, scrolledtext
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
import matplotlib

matplotlib.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei']
matplotlib.rcParams['axes.unicode_minus'] = False

is_running = False
stop_event = threading.Event()
```

```
# 模拟浏览器的会话状态
  实现反爬机制
cookies = {
    'aQQ_ajkguid': '0175308C-6028-4E27-B060-DCD51C97DD86',
    'sessid': 'AA933E31-BEF7-4FEE-A98D-8C1BE6AB56F2',
    'ajk-appVersion': '',
    'id58': 'CkwAhWkMuEq0VjM/UdK7Ag==',
    'xxzlclientid': 'e671c26a-91f2-4716-bec2-1762441571081',
    'xxzlxxid':
'pfmxyMsgPlyAanVusZ74sTHkGpNGCPEjdCt8+zmYiAwnm+xoMo0MM//7d+jU1SvFoEw/',
    '58tj_uuid': 'b9cabf4e-7ef7-4471-a70c-649066743b78',
    'new_uv': '1',
    'ctid': '13',
    'cmctid': '4',
    'twe': '2',
    'fzq_h':
'35c1d323dced0d8cd38d73c7136ef91f_1766459422356_eab9059de5b34519a401d367a738b404_
3730140730',
    'lps':
'https%3A%2F%2Fsz.zu.anjuke.com%2F%3Ffrom%3DHomePage_TopBar%7Chttps%3A%2F%
2Fshenzhen.anjuke.com%2F',
    'fzq_js_anjuke_ershoufang_pc': '0a80a3b6a4fad6d122b91b9ba3d0a863_1766494483843_23',
    'obtain_by': '1',
    'xxzlbbid':
'pfmbRLvd/247FB+6aexjtHWY54FmcKuJNhH8KJzgfc5Hq9ZmIBeTvg1wqgea6GMxiMC1Z6F
O2BNzxn9jDoII+IHFNvU6t8TkB2DOOZB2/yElxX6CblkYugVvGOGe/5ucgriudhoQqZ0xNzY2
NDk0NDg0ODUxMDQw_1',
}

# 构造合法请求头，伪装浏览器访问
headers = {
    'accept':
'text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q
=0.8,application/signed-exchange;v=b3;q=0.7',
    'accept-language':
'zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6,pt-BR;q=0.5,pt;q=0.4',
    'cache-control': 'no-cache',
    'dnt': '1',
    'pragma': 'no-cache',
    'priority': 'u=0, i',
    'referer': 'https://shenzhen.anjuke.com/sale/?from=HomePage_TopBar',
    'sec-ch-ua': '"Microsoft Edge";v="143", "Chromium";v="143", "Not A(Brand";v="24"',
    'sec-ch-ua-mobile': '?0',
```

```
    'sec-ch-ua-platform': '"Windows"',
    'sec-fetch-dest': 'document',
    'sec-fetch-mode': 'navigate',
    'sec-fetch-site': 'same-origin',
    'sec-fetch-user': '?1',
    'upgrade-insecure-requests': '1',
    'user-agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0',
}

# 定义原始数据文件
CSV_FILE = '深圳二手房.csv'
# 清洗过的数据文件
CLEANED_FILE = '深圳二手房_cleaned.csv'
# CSV 文件的表头结构
CSV_HEADERS = ['标题', '户型', '面积', '朝向', '楼层', '建造年份', '小区名称', '区域', '商圈', '地址', '标签',
               '总价(万)', '单价(元/㎡)']


# 检查深圳二手房.csv 是否存在，如果不存在则创建文件，并写入表头（CSV_HEADERS），保证后续追加数据时格式统一。
def init_csv():
    """初始化 CSV 文件，写入表头"""
    if not os.path.exists(CSV_FILE):
        with open(CSV_FILE, 'w', newline='', encoding='utf-8-sig') as f:
            writer = csv.writer(f)
            writer.writerow(CSV_HEADERS)


def save_to_csv(data_list):
    """追加数据到 CSV 文件"""
    with open(CSV_FILE, 'a', newline='', encoding='utf-8-sig') as f:
        writer = csv.writer(f)
        for data in data_list:
            writer.writerow(data)
    return len(data_list)
```

初始化与页面解析准备

首先将传入的 HTML 文本传入 BeautifulSoup，指定 html.parser 解析器，生成可操作的 soup 对象（这是解析 HTML 的基础）。

通过 CSS 选择器 div.property[tongji_tag="fcpc_ersflist_gzcount"]定位页面中所有的房产卡片元素，这些是数据提取的核心载体。

创建空列表 results，用于存储最终提取的结构化房产数据。
遍历房产卡片提取数据（核心循环）
对每个房产卡片 prop，通过 try-except 包裹所有提取逻辑：
异常处理的作用：即使某一套房源的信息提取出错（比如某个标签缺失），也只会跳过该房源，继续解析下一个，避免整个函数因单个数据异常而中断。
逐个字段提取逻辑（按提取顺序）：
标题：定位 class 为 property-content-title-name 的 h3 标签，优先获取标签的 title 属性值（无则为空），并去除首尾空格。
户型：定位 class 为 property-content-info-attribute 的 p 标签，提取该标签下所有 span 子标签的文本并拼接（户型信息通常分散在多个 span 中）。
面积 / 朝向 / 楼层 / 建造年份：先定位第一组 property-content-info-text 类的 p 标签集合，再遍历每个标签的文本：
包含“㎡”的文本判定为面积；
包含“层”的文本判定为楼层；
包含“年建造”的文本判定为建造年份；
匹配指定朝向关键词（南、北、南北等）的文本判定为朝向。
小区名称：定位 class 为 property-content-info-comm-name 的 p 标签，提取其文本并去空格。
详细地址（区域/商圈/具体地址）：定位地址相关的 span 标签集合，按索引分别提取区域（第 0 个）、商圈（第 1 个）、具体地址（第 2 个），并做索引越界保护（避免标签数量不足时报错）。
房源标签：提取所有 class 为 property-content-info-tag 的 span 标签文本，用逗号拼接成字符串（比如“近地铁，精装”）。
总价/单价：分别定位总价、单价对应的标签，提取文本并去空格。
数据过滤与存储
只有当标题 title 不为空时（保证数据有效性，排除无标题的无效房源），才将提取的所有字段按固定顺序组装成列表，添加到 results 中。
返回结果
遍历完所有房产卡片后，返回整理好的 results 列表，列表中每个元素是一套房产的完整信息列表。

```
def parse_page(html):
    """解析页面 HTML，提取房产数据"""
    soup = BeautifulSoup(html, 'html.parser')
    properties = soup.select('div.property[tongji_tag="fcpc_ersflist_gzcount"]')
    results = []

    for prop in properties:
        try:
            # # 提取标题 匹配 class 为 property-content-title-name 的 h3 标签 找 title
              title_elem = prop.select_one('h3.property-content-title-name')
              title = title_elem.get('title', '').strip() if title_elem else ''

              # 户型 p 标签 class 为 property-content-info-attribute
              attr_elem = prop.select_one('p.property-content-info-attribute')
              layout = ''
```

```
            if attr_elem:
                # 父标签内所有的 span 子标签
                spans = attr_elem.select('span')
                # 拼接 span 标签
                layout = ''.join([s.text for s in spans])

            # 面积、朝向、楼层、建造年份
            info_texts = prop.select('div.property-content-info:first-of-type p.property-content-info-text')
            area = ''
            orientation = ''
            floor = ''
            build_year = ''

            for info in info_texts:
                text = info.text.strip()
                if 'm²' in text:
                    area = text
                elif '层' in text:
                    floor = text
                elif '年建造' in text:
                    build_year = text
                elif text in ['南', '北', '东', '西', '南北', '东西', '东南', '东北', '西南', '西北']:
                    orientation = text
            # 小区名称的 p 标签
            comm_name = prop.select_one('p.property-content-info-comm-name')
            community = comm_name.text.strip() if comm_name else ''
            # 详细地址 区域 商圈 地址
            addr_spans = prop.select('p.property-content-info-comm-address span')
            district = addr_spans[0].text.strip() if len(addr_spans) > 0 else ''
            business_area = addr_spans[1].text.strip() if len(addr_spans) > 1 else ''
            address = addr_spans[2].text.strip() if len(addr_spans) > 2 else ''
            # 房源标签
            tags = prop.select('span.property-content-info-tag')
            tag_str = ','.join([t.text.strip() for t in tags])
            # 总价
            total_price = prop.select_one('span.property-price-total-num')
            price_total = total_price.text.strip() if total_price else ''
            # 单价
            avg_price = prop.select_one('p.property-price-average')
            price_avg = avg_price.text.strip() if avg_price else ''

            if title:
                results.append([title, layout, area, orientation, floor, build_year,
```

```
                                        community, district, business_area, address, tag_str, price_total, price_avg])
            except Exception:
                continue

    return results

def clean_data():
    """清洗爬取的数据"""
    if not os.path.exists(CSV_FILE):
        return None, "未找到数据文件"
    # 读取 csv 文件
    df = pd.read_csv(CSV_FILE, encoding='utf-8-sig')

    # 去除重复数据
    # 清洗前的总行数
    original_count = len(df)
    #按「标题+小区名称」组合去重
    df = df.drop_duplicates(subset=['标题', '小区名称'], keep='first')
    #清洗面积列 提取数值 89 ㎡->89.0
    df['面积_数值'] = df['面积'].str.extract(r'(\d+\.?\d*)').astype(float)
    #清洗总价 转换为数值
    df['总价_数值'] = pd.to_numeric(df['总价(万)'], errors='coerce')
    #清洗单价 提取数值
    df['单价_数值'] = df['单价(元/㎡)'].str.extract(r'(\d+)').astype(float)
    # 清洗建造年份 提取年份
    df['建造年份_数值'] = df['建造年份'].str.extract(r'(\d{4})').astype(float)
    # 从户型中提取房间数
    df['房间数'] = df['户型'].str.extract(r'(\d+)室').astype(float)

    #删除缺失关键数据的行
    df = df.dropna(subset=['总价_数值', '面积_数值'])
    # 保存清洗后的数据到 CSV 文件
    df.to_csv(CLEANED_FILE, index=False, encoding='utf-8-sig')
    # 清洗后的有效行数
    cleaned_count = len(df)
    # 被删除的行数
    removed = original_count - cleaned_count

    return df, f"清洗完成: {original_count} -> {cleaned_count} (删除 {removed} 条)"
```

爬取类：

```
class CrawlerApp:
    def __init__(self, root):
```

```
        # 初始化窗口标题 尺寸...
        self.root = root
        self.root.title("深圳二手房数据爬取")
        self.root.geometry("900x700")
        # 记录当前是否正在执行爬取任务 防止重复开始
        self.is_running = False
        # 线程停止信号（优雅停止）
        self.stop_event = threading.Event()
        # 保存爬取线程对象
        self.crawl_thread = None
        # 调用 setupui 构建界面
        self.setup_ui()

    def setup_ui(self):
        # 在父容器 root 上添加控制面板
        control_frame = ttk.LabelFrame(self.root, text="控制面板", padding=10)
        # 控制面板放在界面上
        control_frame.pack(fill=tk.X, padx=10, pady=5)

        # 页码范围输入--起始页 默认为 1
        ttk.Label(control_frame, text="起始页:").grid(row=0, column=0, padx=5)
            # 创建字符串变量，用于绑定输入框内容，默认值为"1"
        self.start_page_var = tk.StringVar(value="1")
          # 创建起始页输入框
        ttk.Entry(control_frame,      textvariable=self.start_page_var,      width=8).grid(row=0,
column=1, padx=5)


        # 页码范围输入--结束页 默认为 10
        ttk.Label(control_frame, text="结束页:").grid(row=0, column=2, padx=5)
        self.end_page_var = tk.StringVar(value="10")
        ttk.Entry(control_frame,      textvariable=self.end_page_var,      width=8).grid(row=0,
column=3, padx=5)

        # 按钮
        # 绑定 start_craw 方法 开始按钮 绑定爬取数据
        self.start_btn = ttk.Button(control_frame, text="开始", command=self.start_crawl)
        self.start_btn.grid(row=0, column=4, padx=10)

        # 创建"停止"按钮，父容器是 control_frame，点击触发 self.stop_crawl 方法
        self.stop_btn = ttk.Button(control_frame, text="停止", command=self.stop_crawl,
state=tk.DISABLED)
        self.stop_btn.grid(row=0, column=5, padx=5)
```

```
        # 创建"数据清洗"按钮，父容器是 control_frame，点击触发 self.clean_data 方法
        self.clean_btn = ttk.Button(control_frame, text="数据清洗", command=self.clean_data)
        self.clean_btn.grid(row=0, column=6, padx=5)
        # 创建"可视化"按钮，父容器是 control_frame，点击触发 self.show_visualization 方法
        self.viz_btn = ttk.Button(control_frame, text="可视化", command=self.show_visualization)
        self.viz_btn.grid(row=0, column=7, padx=5)

        # 进度条
        # 用于绑定进度条的进度值
        self.progress_var = tk.DoubleVar()
        # 设置进度条 ui
        self.progress = ttk.Progressbar(control_frame, variable=self.progress_var, maximum=100)
        self.progress.grid(row=1, column=0, columnspan=8, sticky='ew', pady=10)

        # 状态标签
        # 创建字符串变量，用于绑定状态标签内容，默认值为"就绪
        self.status_var = tk.StringVar(value="就绪")
        ttk.Label(control_frame, textvariable=self.status_var).grid(row=2, column=0, columnspan=8)

        # 日志区域
        # 日志框架
        log_frame = ttk.LabelFrame(self.root, text="日志", padding=10)
        log_frame.pack(fill=tk.BOTH, expand=True, padx=10, pady=5)
        #创建带滚动条的文本框
        self.log_text = scrolledtext.ScrolledText(log_frame, height=15)
        self.log_text.pack(fill=tk.BOTH, expand=True)

        # 可视化区域
        self.viz_frame = ttk.LabelFrame(self.root, text="数据可视化", padding=10)
        self.viz_frame.pack(fill=tk.BOTH, expand=True, padx=10, pady=5)


    def log(self, message):
        """记录日志信息"""
        self.log_text.insert(tk.END, f"{time.strftime('%H:%M:%S')} - {message}\n")
        self.log_text.see(tk.END)
        self.root.update_idletasks()

    def start_crawl(self):
        """开始爬取 准备工作"""
```

```
        # 如果有进程在执行 直接返回
        if self.is_running:
            return

        try:
            # 绑定起始框中的内容
            start_page = int(self.start_page_var.get())
            # 绑定结束框中的内容
            end_page = int(self.end_page_var.get())

            if start_page < 1 or end_page < start_page:
                messagebox.showerror("错误", "页码范围无效")
                return
        except ValueError:
            messagebox.showerror("错误", "请输入有效的页码")
            return

        # 表示当前有进程在运行
        self.is_running = True
        # 清除停止信号
        self.stop_event.clear()
        # 设置显示进度条下方的状态
        self.status_var.set("爬取中...")
        # 禁用开始和结束按钮
        self.start_btn.config(state=tk.DISABLED)
        self.stop_btn.config(state=tk.NORMAL)

        # 创建爬取线程对象
        self.crawl_thread = threading.Thread(target=self.crawl_worker, args=(start_page, end_page))
        # 设置线程为守护线程，确保程序退出时线程能随主程序一起终止
        self.crawl_thread.daemon = True
        # 启动线程，开始执行 self.crawl_worker()方法中的爬取逻辑
        self.crawl_thread.start()

    def stop_crawl(self):
        """停止爬取"""
        self.stop_event.set()
        self.status_var.set("正在停止...")
        self.log("已发送停止信号，等待当前页面完成...")

    def crawl_worker(self, start_page, end_page):
        """爬取工作线程"""
        # 初始化 不存在则创建+写表头
```

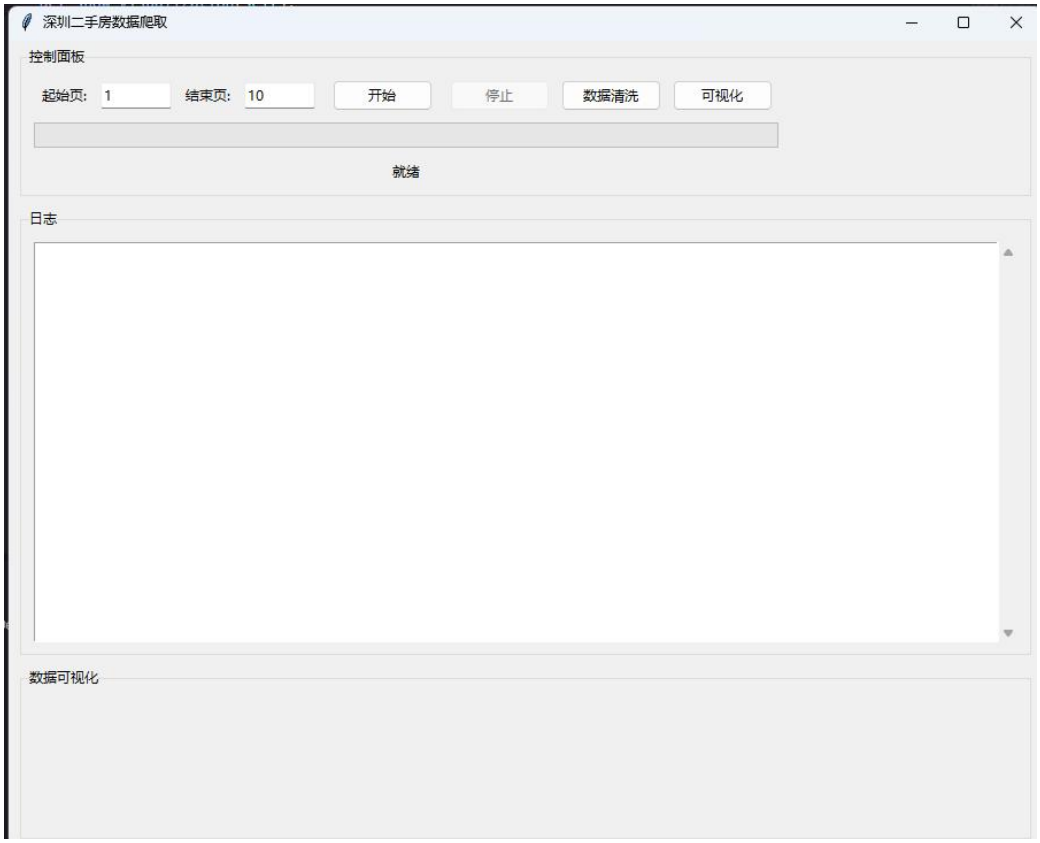
```
init_csv()
# 计算总爬取页数（如 1-10 页共 10 页）
total_pages = end_page - start_page + 1
# 初始化总保存记录数，用于最终汇总
total_saved = 0

# 循环爬取每一页
for page in range(start_page, end_page + 1):
    # 子步骤 1：检查停止信号，实现优雅终止
    if self.stop_event.is_set():
        # 线程安全记录日志：用户主动停止爬取
        self.root.after(0, lambda: self.log("用户停止爬取"))
        break

    progress = ((page - start_page) / total_pages) * 100
    self.root.after(0, lambda p=progress: self.progress_var.set(p))
    # 记录日志
    self.root.after(0, lambda pg=page: self.log(f"正在爬取第 {pg} 页..."))

    if page == 1:
        url = 'https://shenzhen.anjuke.com/sale/'
    else:
        url = f'https://shenzhen.anjuke.com/sale/p{page}/'

    # 爬取网页
    # 携带 cookie 和 headers 伪装浏览器
    try:
        response = requests.get(url, params={'from': 'HomePage_TopBar'},
                                cookies=cookies, headers=headers, timeout=10)
        # 检查是否成功
        response.raise_for_status()
        # 解析网页源码：调用 parse_page()提取二手房数据，返回数据列表
        data_list = parse_page(response.text)
        # 如果解析到数据
        if data_list:
            # 保存数据到 CSV：调用 save_to_csv()，返回当前页保存的记录数
            count = save_to_csv(data_list)
            total_saved += count

            self.root.after(0, lambda c=count: self.log(f"  已保存 {c} 条记录"))
        # 如果未解析到数据
        else:
            self.root.after(0, lambda: self.log("  未找到数据，可能已到最后一页"))
            break
```

```
                time.sleep(2)

            except Exception as e:
                self.root.after(0, lambda err=str(e): self.log(f"    错误: {err}"))
                continue
        # 爬取结束
        # 线程安全更新进度条到 100%（无论是否爬完，都拉满进度）
        self.root.after(0, lambda: self.progress_var.set(100))
        # 线程安全记录总结果：共保存多少条记录
        self.root.after(0, lambda t=total_saved: self.log(f"爬取完成，共保存 {t} 条记录"))
        # 爬取完成回调，重置
        self.root.after(0, self.crawl_finished)

    def crawl_finished(self):
        """爬取完成回调"""
        # 重置爬取状态标记，允许再次启动爬取
        self.is_running = False
        # 启用「开始」按钮，恢复可点击状态
        self.start_btn.config(state=tk.NORMAL)
        #禁用「停止」按钮，恢复初始状态
        self.stop_btn.config(state=tk.DISABLED)
        # 更新状态标签，告知用户爬取任务结束
        self.status_var.set("已完成")

    def clean_data(self):
        """清洗数据"""
        self.log("正在清洗数据...")
        # 调用全局 clean_data() 函数，执行实际清洗逻辑
        df, message = clean_data()
        # 清洗操作成功完成
        if df is not None:
            self.log(message)
            self.log(f"清洗后数据已保存到 {CLEANED_FILE}")
            messagebox.showinfo("成功", message)
        else:
            self.log(message)
            messagebox.showerror("错误", message)


    def show_visualization(self):

        # 清除之前的可视化 重置展示区域
        for widget in self.viz_frame.winfo_children():
```

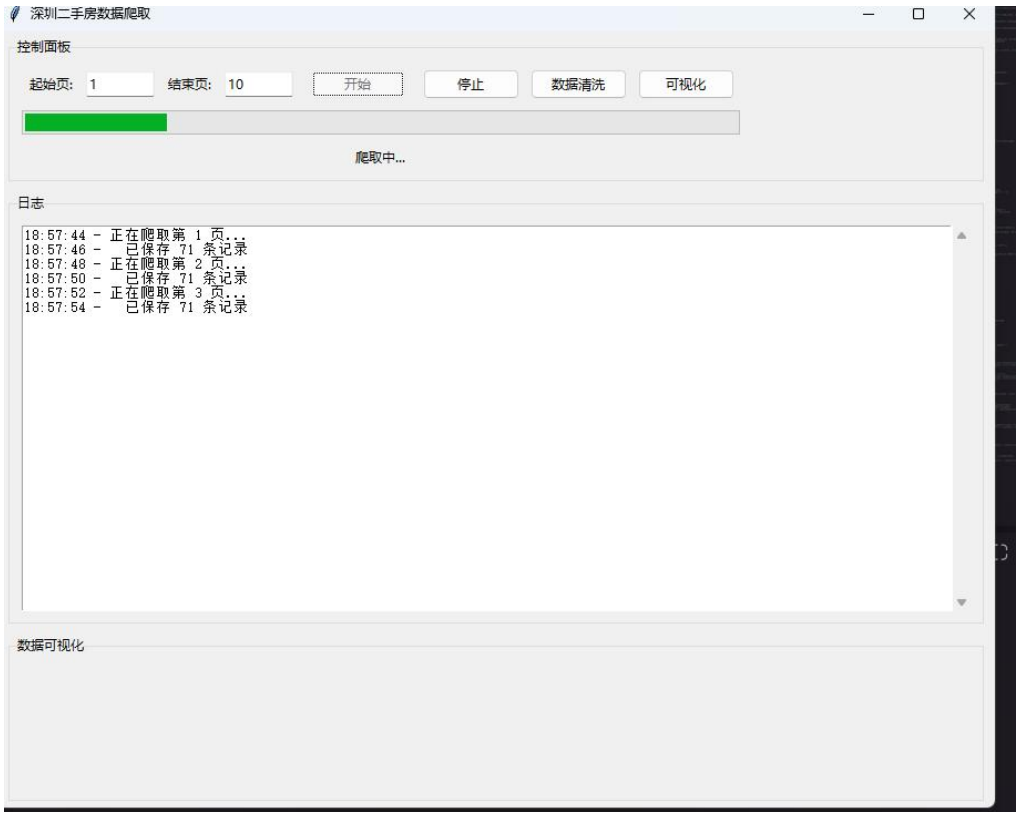
```
        widget.destroy()

    # 优先使用清洗后的数据文件
    data_file = CLEANED_FILE if os.path.exists(CLEANED_FILE) else CSV_FILE
    # 未找到
    if not os.path.exists(data_file):
        messagebox.showerror("错误", "未找到数据文件，请先爬取数据")
        return

    try:
        # 读取数据并统一预处理
        df = pd.read_csv(data_file, encoding='utf-8-sig')
        if len(df) == 0:
            messagebox.showerror("错误", "数据文件为空")
            return

        # 统一数据预处理（确保字段格式一致） 无论读取的是【原始数据】还是【清洗后数据】，都统一生成绘图所需的「纯数值字段」，保证数据格式一致
        df['总价_数值'] = pd.to_numeric(df['总价(万)'], errors='coerce') if '总价_数值' not in df.columns else df[
            '总价_数值']
        df['面积_数值'] = df['面积'].str.extract(r'(\d+\.?\d*)').astype(float) if '面积_数值' not in df.columns else \
        df['面积_数值']
        df['单价_数值'] = df['单价(元/㎡)'].str.extract(r'(\d+)').astype(float) if '单价_数值' not in df.columns else \
        df['单价_数值']
        df['房间数'] = df['户型'].str.extract(r'(\d+)室').astype(float) if '房间数' not in df.columns else df[
            '房间数']

        self.log(f"正在加载可视化，数据来源: {data_file} ({len(df)} 条记录)")

        # 创建画布 初始化画布
        plt.figure(figsize=(12, 10))
        # 调整子图的布局间距
        plt.subplots_adjust(left=0.1, right=0.95, top=0.95, bottom=0.08, hspace=0.35, wspace=0.3)

        # 1. 各区域平均房价（第一个子图 2 行 2 列第 1 个位置）
        plt.subplot(2, 2, 1)
        # 按「区域」分组，计算每个区域的房源平均总价 按均价从高到低排序 取前十
        district_price = df.groupby('区域')['总价_数值
```

```
'].mean().sort_values(ascending=False).head(10)
            # 绘制横向柱状图
            plt.barh(district_price.index, district_price.values, color='steelblue')
            # x 轴标签
            plt.xlabel('平均价格 (万)', fontsize=10)
            plt.title('各区域平均房价', fontsize=12, pad=7)
            plt.tick_params(axis='y', labelsize=9)

            # 2. 户型分布（第二个子图 2 行 2 列第 2 个位置）
            plt.subplot(2, 2, 2)
            # 每个房间数对应的房源数量（如 3 室有 120 套、2 室有 80 套），返回 Series
            # 房间数「从小到大」排序 过滤掉「房间数为空」的无效数据
            room_counts = df['房间数'].value_counts().sort_index().dropna()
            plt.bar(room_counts.index.astype(str), room_counts.values, color='coral')
            plt.xlabel('房间数', fontsize=10)
            plt.ylabel('数量', fontsize=10)
            plt.title('户型分布', fontsize=12, pad=7)

            # 3. 价格与面积散点图（第三个子图 2 行 2 列第 3 个位置）
            plt.subplot(2, 2, 3)
            # 删除「面积或总价为空」的房源 数据量 > 500 条：随机采样 500 条
            sample_df = df.dropna(subset=['面积_数值', '总价_数值']).sample(min(500,
len(df)))
            # 设置 xy 轴
            plt.scatter(sample_df['面积_数值'], sample_df['总价_数值'], alpha=0.5, s=10)
            plt.xlabel('面积 (㎡)', fontsize=10)
            plt.ylabel('总价 (万)', fontsize=10)
            plt.title('价格与面积关系', fontsize=12, pad=7)

            # 4. 单价分布（第四个子图 2 行 2 列第 4 个位置）
            plt.subplot(2, 2, 4)
            # DataFrame 中提取「单价_数值」列，并通过.dropna()删除所有空值（NaN），
得到纯数值的 Series
            price_data = df['单价_数值'].dropna()
            # 清洗后的单价数值 价数值分成「30 个等宽区间」
            plt.hist(price_data, bins=30, color='green', alpha=0.7, edgecolor='black')
            plt.xlabel('单价 (元/㎡)', fontsize=10)
            plt.ylabel('数量', fontsize=10)
            plt.title('单价分布', fontsize=12, pad=7)

            # 获取当前 figure 并嵌入到 tkinter
            fig = plt.gcf()  # 获取当前的 figure 对象
            canvas = FigureCanvasTkAgg(fig, master=self.viz_frame)
            canvas.draw()
```

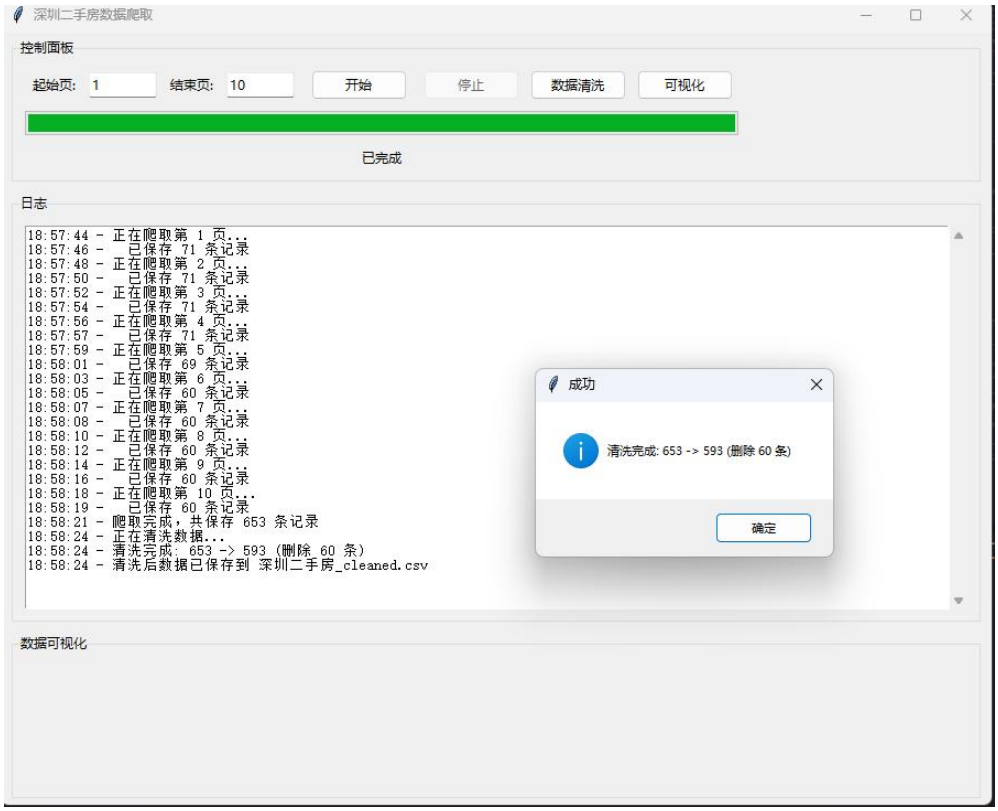
```
            canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)

            self.log("可视化生成成功")

        except Exception as e:
            self.log(f"可视化错误: {str(e)}")
            messagebox.showerror("错误", f"生成可视化失败: {str(e)}")


def main():
    # 根窗口对象 root
    root = tk.Tk()
    app = CrawlerApp(root)
    root.mainloop()


if __name__ == '__main__':
    main()
```



初始界面


深圳二手房数据爬取
控制面板
起始页: 1
结束页: 10
开始
停止
数据清洗
可视化
爬取中...
日志
18:57:44 - 正在爬取第 1 页...
18:57:46 - 已保存 71 条记录
18:57:48 - 正在爬取第 2 页...
18:57:50 - 已保存 71 条记录
18:57:52 - 正在爬取第 3 页...
18:57:54 - 已保存 71 条记录
数据可视化

爬取过程界面


深圳二手房数据爬取
控制面板
起始页: 1
结束页: 10
开始
停止
数据清洗
可视化
已完成
日志
18:57:44 - 正在爬取第 1 页...
18:57:46 - 已保存 71 条记录
18:57:48 - 正在爬取第 2 页...
18:57:50 - 已保存 71 条记录
18:57:52 - 正在爬取第 3 页...
18:57:54 - 已保存 71 条记录
18:57:56 - 正在爬取第 4 页...
18:57:57 - 已保存 71 条记录
18:57:59 - 正在爬取第 5 页...
18:58:01 - 已保存 69 条记录
18:58:03 - 正在爬取第 6 页...
18:58:05 - 已保存 60 条记录
18:58:07 - 正在爬取第 7 页...
18:58:08 - 已保存 60 条记录
18:58:10 - 正在爬取第 8 页...
18:58:12 - 已保存 60 条记录
18:58:14 - 正在爬取第 9 页...
18:58:16 - 已保存 60 条记录
18:58:18 - 正在爬取第 10 页...
18:58:19 - 已保存 60 条记录
18:58:21 - 爬取完成，共保存 653 条记录
18:58:24 - 正在清洗数据...
18:58:24 - 清洗完成: 653 -> 593 (删除 60 条)
18:58:24 - 清洗后数据已保存到 深圳二手房_cleaned.csv
成功
清洗完成: 653 -> 593 (删除 60 条)
确定
数据可视化

数据清洗界面

可视化界面

3.2 分析结果

通过可视化图表得出以下核心分析结果：

3.2.1 区域房价分析

南山区、福田区的平均房价位居前两位，均超过 600 万/套，体现了深圳核心城区的房产价值优势。

坪山区、光明区等远郊区县的平均房价较低，多在 400 万/套以下，价格梯度明显。

区域房价差异主要受地理位置、配套设施、产业分布等因素影响。

3.2.2 户型分布分析

3 室户型的房源数量最多，占比约 30%，是深圳二手房市场的主流户型，符合家庭居住需求。

2 室和 4 室户型占比约 10%，主要面向刚需购房者和小家庭用户。

1 室及五室以上户型占比较低，分别满足单身用户和改善型用户的需求。

3.2.3 价格与面积相关性分析

散点图呈现正相关趋势，即房源面积越大，总价越高，符合房地产市场的基本规律。

80-100 ㎡的房源总价集中在 500-800 万区间，是市场交易的热门面积段。

部分房源存在价格异常点，可能与房屋的装修情况、楼层、地理位置等附加因素相关。

3.2.4 单价分布分析

深圳二手房单价主要集中在 5-9 万/㎡区间，占比约 60%，是市场的主流单价范围。

单价超过 12 万/㎡的房源占比较低，主要集中在核心城区的优质地段。

单价分布呈正态分布特征，说明深圳二手房市场价格相对稳定，未出现极端分化。

4. 项目总结

4.1 项目亮点

功能完整性：实现了从数据爬取、存储、清洗到可视化的全流程自动化处理，满足用户多样化需求。

稳定性强：通过请求伪装、访问频率控制、异常处理等机制，有效绕过网站反爬限制，保证爬取过程稳定可靠。

用户体验佳：基于 tkinter 构建直观的交互界面，操作简单易懂，日志记录和进度条实时反馈系统状态。

数据质量高：完善的数据清洗流程，确保分析数据的准确性和有效性，为可视化结果提供可靠支撑。

扩展性好：代码模块化设计，便于后续功能扩展（如新增数据导出格式、增加更多统计图表等）。

4.2 存在不足

爬取范围有限，仅支持安居客深圳二手房列表页数据，未覆盖详情页的更多字段（如装修情况、物业费等）。

反爬机制应对能力有限，若网站更新反爬策略（如增加验证码、动态加载等），系统可能无法正常爬取。

可视化图表类型相对基础，缺乏 3D 图表、动态交互图表等更丰富的展示形式。

数据存储仅支持 CSV 格式，未提供数据库存储选项，不便于大规模数据管理。

4.3 改进方向

扩展爬取功能：增加详情页爬取逻辑，提取更多房源字段；支持多城市数据爬取，提升系统通用性。

强化反爬能力：引入 cookie 模拟浏览器动态加载，应对复杂反爬机制；添加 IP 代理池，避免 IP 被封禁。

丰富可视化效果：集成更多 matplotlib 图表类型，如散点图、柱状图等；增加图表交互功能（如点击查看房源详情）。

优化数据存储：支持 MySQL、SQLite 等数据库存储，实现数据的高效查询和管理；增加数据备份和恢复功能。

提升界面美观度：使用 tkinter 的美化库（如 ttkbootstrap）优化界面样式，提升视觉体验。

4.4 学习心得

通过本次项目开发，我深入掌握了 Python 爬虫、数据处理、桌面应用开发等核心技术，具体收获如下：

熟练运用 requests 和 BeautifulSoup 库实现网页数据爬取，理解了 HTTP 请求、HTML 解析的核心原理。

掌握了 pandas 库的数据处理方法，包括数据读取、清洗、转换、统计等关键操作。

学会了使用 matplotlib 库生成各类统计图表，并实现图表与 tkinter 界面的嵌入集成。

理解了多线程编程的基本原理，能够通过线程控制实现爬取任务与 UI 交互的分离，避免界面阻塞。

培养了问题解决能力，在应对网站反爬、数据格式异常、界面交互问题等过程中，学会了分析问题并找到有效解决方案。